

YOLOV8模型部署

1.模型准备

1.1 创建YOLOV8环境

使用conda创建yolov8项目环境，输入：

```
conda create -n py39_yolov8 python=3.9
```

创建完成后激活环境并安装依赖包：

```
conda activate py39_yolov8
pip3 install torch==2.4.1 torchvision==0.19.1 torchaudio==2.4.1
pip install ultralytics==8.3.31 onnx==1.17.0 onnxruntime==1.19.2 onnxsim==0.4.36
numpy==1.26.4
```

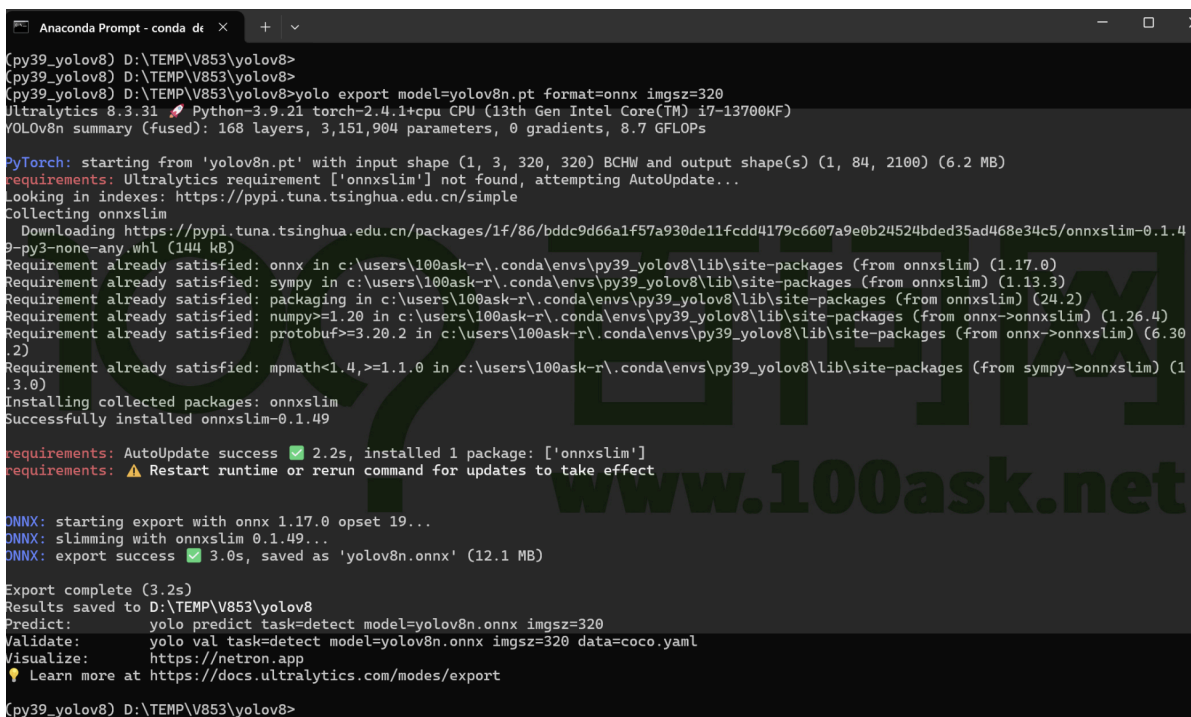
新建yolov8工作目录，下载[yolov8n.pt](#)模型文件。

- YOLOV8n V8.2.0模型文件： [yolov8n.pt](#)
- YOLOV8s V8.2.0模型文件： [yolov8s.pt](#)
- YOLOV8 V8.2.0源码： [ultralytics/ultralytics at v8.2.0](#)

1.2 转换为ONNX模型

将pytorch模型转换为onnx模型，并支持 320x320 像素的图像输入，在终端下执行：

```
yolo export model=yolov8n.pt format=onnx imgsz=320
```



```
Anaconda Prompt - conda de x + v
(py39_yolov8) D:\TEMP\V853\yolov8>
(py39_yolov8) D:\TEMP\V853\yolov8>
(py39_yolov8) D:\TEMP\V853\yolov8> yolo export model=yolov8n.pt format=onnx imgsz=320
Ultralytics 8.3.31 Python-3.9.21 torch-2.4.1+cpu CPU (13th Gen Intel Core(TM) i7-13700KF)
YOLOv8n summary (fused): 168 layers, 3,151,904 parameters, 0 gradients, 8.7 GFLOPs

PyTorch: starting from 'yolov8n.pt' with input shape (1, 3, 320, 320) BCHW and output shape(s) (1, 84, 2100) (6.2 MB)
requirements: Ultralytics requirement ['onnxslim'] not found, attempting AutoUpdate...
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Collecting onnxslim
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/1f/86/bddc9d66a1f57a930de11fcdd4179c6607a9e0b24524bde35ad468e34c5/onnxslim-0.1.4
9-py3-none-any.whl (144 kB)
Requirement already satisfied: onnx in c:\users\100ask-r\.conda\envs\py39_yolov8\lib\site-packages (from onnxslim) (1.17.0)
Requirement already satisfied: sympy in c:\users\100ask-r\.conda\envs\py39_yolov8\lib\site-packages (from onnxslim) (1.13.3)
Requirement already satisfied: packaging in c:\users\100ask-r\.conda\envs\py39_yolov8\lib\site-packages (from onnxslim) (24.2)
Requirement already satisfied: numpy>=1.20 in c:\users\100ask-r\.conda\envs\py39_yolov8\lib\site-packages (from onnx->onnxslim) (1.26.4)
Requirement already satisfied: protobuf>=3.20.2 in c:\users\100ask-r\.conda\envs\py39_yolov8\lib\site-packages (from onnx->onnxslim) (6.30
.2)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in c:\users\100ask-r\.conda\envs\py39_yolov8\lib\site-packages (from sympy->onnxslim) (1
.3.0)
Installing collected packages: onnxslim
Successfully installed onnxslim-0.1.49

requirements: AutoUpdate success 2.2s, installed 1 package: ['onnxslim']
requirements: ⚠ Restart runtime or rerun command for updates to take effect

ONNX: starting export with onnx 1.17.0 opset 19...
ONNX: slimming with onnxslim 0.1.49...
ONNX: export success 3.0s, saved as 'yolov8n.onnx' (12.1 MB)

Export complete (3.2s)
Results saved to D:\TEMP\V853\yolov8
Predict:      yolo predict task=detect model=yolov8n.onnx imgsz=320
Validate:     yolo val task=detect model=yolov8n.onnx imgsz=320 data=coco.yaml
Visualize:    https://netron.app
💡 Learn more at https://docs.ultralytics.com/modes/export
(py39_yolov8) D:\TEMP\V853\yolov8>
```

执行完成后可在当前目录看到生成的 yolov8n.onnx 模型文件。

1.3 简化模型

由于简化模型可以移除冗余节点（如 shape、reshape 链条），加快推理速度，可输入：

```
python -m onnxsim yolov8n.onnx yolov8n-sim.onnx --input-shape 1,3,320,320
```

```
(py39_yolov8) D:\TEMP\V853\yolov8>python -m onnxsim yolov8n.onnx yolov8n-sim.onnx --input-shape 1,3,320,320
WARNING: "--input-shape" is renamed to "--overwrite-input-shape". Please use it instead.
Simplifying...
Finish! Here is the difference:
```

	Original Model	Simplified Model
Add	8	8
Concat	19	19
Constant	144	143
Conv	64	64
Div	1	1
MaxPool	3	3
Mul	58	58
Reshape	5	5
Resize	2	2
Sigmoid	58	58
Slice	2	2
Softmax	1	1
Split	9	9
Sub	2	2
Transpose	1	1
Model Size	12.1MiB	12.1MiB

简化完成后可以得到 yolov8n-sim.onnx 模型。

2.转换NPU模型

2.1 创建转换目录

打开NPU工具包的虚拟机Ubuntu20.04，创建yolov8-8.2文件夹，存放模型和量化图像等。

```
ubuntu@ubuntu2004:~$ mkdir yolov8-8.2
```

进入yolov5模型转换目录。

```
ubuntu@ubuntu2004:~$ cd yolov8-8.2/
```

创建data目录存放量化图像

```
mkdir data
```

将量化图像传入data文件夹下，例如，传入 test01.jpg 图像到 data

```
ubuntu@ubuntu2004:~/yolov8-8.2$ ls data
test01.jpg
```

在yolov5模型转换目录中创建 dataset.txt

```
ubuntu@ubuntu2004:~/yolov8-8.2$ touch dataset.txt
```

修改 dataset.txt 文件

```
ubuntu@ubuntu2004:~/yolov8-8.2$ vi dataset.txt
```

在 `dataset.txt` 文件中增加量化图片的路径。

```
./data/test01.jpg
```

将 `yolov8n-sim.onnx` 模型传入 `yolov8` 模型转换文件夹下。例如：

```
ubuntu@ubuntu2004:~/yolov8-8.2$ ls
data  dataset.txt  yolov8n-sim.onnx
```

工作目录的文件如下所示：

```
ubuntu@ubuntu2004:~/yolov8-8.2$ tree
.
├── data
│   └── test01.jpg
├── dataset.txt
└── yolov8n-sim.onnx

1 directory, 3 files
```

2.2 导入模型

```
pegasus import onnx --model yolov8n-sim.onnx --output-data yolov8n-sim.data --
output-model yolov8n-sim.json
```

导入生成两个文件，分别是 `yolov8n-sim.data` 和 `yolov8n-sim.json` 文件，两个文件是 YOLO V8 网络对应的芯原内部格式表示文件，`data` 文件储存权重，`cfg` 文件储存模型。

2.3 生成 YML 文件

YML 文件对网络的输入和输出的超参数进行描述以及配置，这些参数包括，输入输出 tensor 的形状，归一化系数（均值，零点），图像格式，tensor 的输出格式，后处理方式等等

```
pegasus generate inputmeta --model yolov8n-sim.json --input-meta-output yolov8n-
sim_inputmeta.yml
```

```
ubuntu@ubuntu2004:~/yolov8-8.2$ pegasus generate inputmeta --model yolov8n-sim.json --input-meta-output yolov8n-sim_inputmeta.yml
2023-06-19 06:13:34.277071: W tensorflow/stream_executor/platform/default/dso_loader.cc:59] Could not load dynamic library 'libcudart.so.10.1'; dlerror: libcudart.so.10.1: cannot op
an shared object file: No such file or directory; LD_LIBRARY_PATH: /home/ubuntu/VeriSilicon/acuity-toolkit-binary-6.6.1/bin/acuitylib
2023-06-19 06:13:34.277199: I tensorflow/stream_executor/cuda/cudart_stub.cc:29] Ignore above cudart dlerror if you do not have a GPU set up on your machine.
b Get binary package acuity_path [/home/ubuntu/VeriSilicon/acuity-toolkit-binary-6.6.1/bin]
i Namespace(generates='inputmeta', input_meta_output='yolov8n-sim_inputmeta.yml', model='yolov8n-sim.json', separated_database=False, which='generate')
b Get binary package acuity_path [/home/ubuntu/VeriSilicon/acuity-toolkit-binary-6.6.1/bin]
b Load model in yolov8n-sim.json
i Generate input meta yolov8n-sim_inputmeta.yml
i .....Error(e).Warning(e).....
```

```
pegasus generate postprocess-file --model yolov8n-sim.json --postprocess-file-
output yolov8n-sim_postprocess_file.yml
```

修改 `yolov8n-sim_inputmeta.yml` 文件中的 `scale` 参数为 `0.0039216(1/255)`，目的是对输入 `tensor` 进行归一化，和网络进行训练的时候是对应的。

```
vi yolov8n-sim_inputmeta.yml
```

```
(py39_yolov8) D:\TEMP\V853\yolov8>python -m onnxsim yolov8n.onnx yolov8n-sim.onnx --input-shape 1,3,320,320
WARNING: "--input-shape" is renamed to "--overwrite-input-shape". Please use it instead.
Simplifying...
Finish! Here is the difference:
```

	Original Model	Simplified Model
Add	8	8
Concat	19	19
Constant	144	143
Conv	64	64
Div	1	1
MaxPool	3	3
Mul	58	58
Reshape	5	5
Resize	2	2
Sigmoid	58	58
Slice	2	2
Softmax	1	1
Split	9	9
Sub	2	2
Transpose	1	1
Model Size	12.1MiB	12.1MiB

2.4 UINT8量化

生成量化表文件，使用非对称量化，uint8，修改 `--batch-size` 参数为你的 `dataset.txt` 里提供的图片数量。

```
pegasus quantize --model yolov8n-sim.json --model-data yolov8n-sim.data --batch-size 1 --device CPU --with-input-meta yolov8n-sim_inputmeta.yml --rebuild --model-quantize yolov8n-sim.quantize --quantizer asymmetric_affine --qtype uint8
```

```
Quantize tensor(@Reshape_/model.22/Reshape_2_25_acuity_mark_perm_242:out0) with tensor(@Concat_/model.22/Concat_2_28:out0)
D Quantize tensor(@Reshape_/model.22/Reshape_2_25_acuity_mark_perm_243:out0) with tensor(@Reshape_/model.22/Reshape_2_25_acuity_mark_perm_243:out0)
D Quantize tensor(@Reshape_/model.22/dfl/Reshape_20:out0) with tensor(@Split_/model.22/Split_21:out0)
D Quantize tensor(@Softmax_/model.22/dfl/Softmax_18_acuity_mark_perm_240:out0) with tensor(@Reshape_/model.22/dfl/Reshape_20:out0)
D Quantize tensor(@Reshape_/model.22/dfl/Reshape_1_13_acuity_mark_perm_239:out0) with tensor(@Conv_/model.22/dfl/conv/Conv_17:out0)
D Quantize tensor(@Reshape_/model.22/dfl/Reshape_1_13:out0) with tensor(@Reshape_/model.22/dfl/Reshape_1_13_acuity_mark_perm_239:out0)
D Quantize tensor(@Slice_/model.22/Slice_12:out0) with tensor(@Reshape_/model.22/dfl/Reshape_1_13:out0)
D Quantize tensor(@Slice_/model.22/Slice_1_15:out0) with tensor(@Reshape_/model.22/dfl/Reshape_1_13:out0)
D Quantize tensor(@attach_Concat_/model.22/Concat_5/out0_0:out0) with tensor(@Concat_/model.22/Concat_5_1:out0)
I End quantization...
I Dump net quantize tensor table to yolov8n-sim.quantize
I Save net to yolov8n-sim.data
I -----Error(0),Warning(0)-----
ubuntu@ubuntu2004:~/work/yolov8$
```

2.5 预推理

利用前文的量化表执行预推理，得到推理 `tensor`

```
pegasus inference --model yolov8n-sim.json --model-data yolov8n-sim.data --batch-size 1 --dtype quantized --model-quantize yolov8n-sim.quantize --device CPU --with-input-meta yolov8n-sim_inputmeta.yml --postprocess-file yolov8n-sim_postprocess_file.yml
```

```
W:tensorflow:AutoGraph could not transform <acuitylib.core.execution.BackendFunction object at 0x7f795704b400>
and will run it as-is.
Please report this to the TensorFlow team. When filing the bug, set the verbosity to 10 (on Linux, `export AUTO
GRAPH_VERBOSITY=10`) and attach the full output.
Cause: Unable to locate the source code of <function BackendFunction.__call__ at 0x7f7957d850d0>. Note that fun
ctions defined in certain environments, like the interactive Python shell do not expose their source code. If t
hat is the case, you should define them in a .py source file. If you are certain the code is graph-compatibl
e, wrap the call using @tf.autograph.do_not_convert. Original error: could not get source code
To silence this warning, decorate the function with @tf.autograph.experimental.do_not_convert
I Running 1 iterations
I Iter(0), top(5), tensor(@attach_Concat_/model.22/Concat_5/out0_0:out0) :
I 5810: 415.7803955078125
I 5811: 407.5633850097656
I 5809: 386.1991882324219
I 5812: 377.982177734375
I 5829: 371.4085693359375
I Save tensor /home/ubuntu/work/yolov8/iter_0_attach_Concat_model.22_Concat_5_out0_0_out0_1_84_2100.tensor
I Save tensor /home/ubuntu/work/yolov8/iter_0_images_238_out0_1_3_320_320.tensor
I Check const pool...
I End inference...
I -----Error(0),Warning(0)-----
ubuntu@ubuntu2004:~/work/yoLov8$
```

2.6 导出模板代码与模型

输出的模型可以在 `ovx11b/yolov8n-sim_nbg_unify` 文件夹中找到 `network_binary.nb` 文件。

```
pegasus export ovxlib --model yolov8n-sim.json --model-data yolov8n-sim.data --
dtype quantized --model-quantize yolov8n-sim.quantize --batch-size 1 --save-
fused-graph --target-ide-project 'linux64' --with-input-meta yolov8n-
sim_inputmeta.yml --output-path ovxlib/yolov8n-sim/yolov8n-simprj --pack-nbg-
unify --postprocess-file yolov8n-sim_postprocessmeta.yml --optimize
"VIP9000PICO_PID0XEE" --viv-sdk ${VIV_SDK}
```

```

x86_64-linux-gnu-gcc -fcommon -fPIE -pie -Wl,-zrelro,-znow -L/usr/lib/x86_64-linux-gnu -L/home/ubuntu/.local/share/conda/envs/virtualenv/bin/../lib/python3.7/site-packages/vxtutils/...
peg_a -o gen_nbg
[Galcore]: system reserved pool0 CPU physical=0x1000000 GPU physical=0x1000000 virtual=0x1000000 size=0xf3334000
Create Neural Network: 54ms or 54899us
Verify...
Verify Graph: 6441ns or 6441203us
Start run graph [1] times...
Run the 1 timer: 5.60ms or 5060.13us
vxProcessGraph execution time:
Total 5.11ms or 5109.55us
Average 5.11ms or 5109.55us
--- Top5 ---
0: 0.000000
1: 0.000000
2: 0.000000
3: 0.000000
4: 0.000000
D Get binary package acuity_path [/home/ubuntu/VeriSilicon/acuity-toolkit-binary-6.9.3/bin]
D Get binary package acuity_path [/home/ubuntu/VeriSilicon/acuity-toolkit-binary-6.9.3/bin]
D Get binary package acuity_path [/home/ubuntu/VeriSilicon/acuity-toolkit-binary-6.9.3/bin]
D Get binary package acuity_path [/home/ubuntu/VeriSilicon/acuity-toolkit-binary-6.9.3/bin]
Export ovxlib case with version 1.1.34
I Dump nbg input meta to /home/ubuntu/work/yoloV8/ovxlib/yoloV8-sim_nbg_unify/nbg_meta.json
I Save vx network source file to /home/ubuntu/work/yoloV8/ovxlib/yoloV8-sim_nbg_unify/vnn_yoloV8nsimprj.c
I Save vx network source file to /home/ubuntu/work/yoloV8/ovxlib/yoloV8-sim_nbg_unify/vnn_yoloV8nsimprj.h
I Save vx network source file to /home/ubuntu/work/yoloV8/ovxlib/yoloV8-sim_nbg_unify/vnn_post_process.c
I Save vx network source file to /home/ubuntu/work/yoloV8/ovxlib/yoloV8-sim_nbg_unify/vnn_post_process.h
I Save vx network source file to /home/ubuntu/work/yoloV8/ovxlib/yoloV8-sim_nbg_unify/vnn_pre_process.c
I Save vx network source file to /home/ubuntu/work/yoloV8/ovxlib/yoloV8-sim_nbg_unify/vnn_pre_process.h
I Save vx network source file to /home/ubuntu/work/yoloV8/ovxlib/yoloV8-sim_nbg_unify/vnn_global.h
I Save vx network source file to /home/ubuntu/work/yoloV8/ovxlib/yoloV8-sim_nbg_unify/main.c
I Save vx network source file to /home/ubuntu/work/yoloV8/ovxlib/yoloV8-sim_nbg_unify/BUILD
I Save vx network source file to /home/ubuntu/work/yoloV8/ovxlib/yoloV8-sim_nbg_unify/yoloV8nsimprj.vcxproj
I Save vx network source file to /home/ubuntu/work/yoloV8/ovxlib/yoloV8-sim_nbg_unify/yoloV8nsimprj.2012.vcxproj
I Save vx network source file to /home/ubuntu/work/yoloV8/ovxlib/yoloV8-sim_nbg_unify/makefile.linux
I Save vx network source file to /home/ubuntu/work/yoloV8/ovxlib/yoloV8-sim_nbg_unify/project
I Dump net to ovxlib/yoloV8-sim_nbg_unify/yoloV8nsimprj_fused.json
I End exporting ovxlib case...
W -----Error(0),Warning(2)-----
ubuntu@ubuntu2004:~/work/yoloV8$
```

将生成的 `network_binary.nb` 文件拷贝出来备用。

3.端侧部署

4.1 编译测试程序

拷贝配套源码中的 `yolov8-npu-code/yolov8.tar.gz` 至SDK中 `tina-v853-100ask/package/allwinner` 目录下。解压压缩包:

```
tar -xzvf yolov8.tar.gz
```

解压后可得到端侧部署源码。

回到tina-v853-100ask目录下，重新激活环境：

```
ubuntu@ubuntu1804:tina-v853-100ask$ source build/envsetup.sh
...
ubuntu@ubuntu1804:tina-v853-100ask$ lunch

You're building on Linux

Lunch menu... pick a combo:
    1  v853-100ask-tina
    2  v853-vision-tina
which would you like? [Default v853-100ask]: 1
...
```

进入Tina配置界面

```
make menuconfig
```

进入Allwinner目录，选中yolov8后即可编译端侧部署程序。

保存并退出Tina配置界面。

编译Tina SDK镜像，编译完成后打包生成镜像

```
ubuntu@ubuntu1804:tina-v853-100ask$ make
...
ubuntu@ubuntu1804:tina-v853-100ask$ pack
```

如果想单独编译resnet50端侧部署源码可执行：

```
make package/allwinner/yolov8/install
```

编译生成的可执行程序在：~/tina-v853-100ask/out/v853-100ask/compile_dir/target/yolov8/yolov8

编译完成后可使用adb将可执行程序传输至开发板端：

```
adb push ~/tina-v853-100ask/out/v853-100ask/compile_dir/target/yolov8/yolov8
/mnt/UDISK
```

4.2 运行测试程序

提前将yolov8可执行程序、network_binary.nb模型、测试图像(320*320pix)传入开发板端，在开发板执行

```
./yolov8 ./network_binary.nb ./test.jpg
```

```

root@TinaLinux:/mnt/UDISK# ./yolov8 ./network_binary.nb ./bus_320-320.jpg
[34968.757491] npu[c13][c13] vipcore, device init..
[34968.762875] set_vip_power_clk ON
[34968.768584] enter aw vip mem alloc size 10485760 vip_heap
[34968.800073] aw_vip_mem_alloc vir 0xe31fe000, phy 0x48800000
[34968.806447] npu[c13][c13] gckvip_drv_init kernel logical phy address=0x48800000 virtual =0xe31fe000
Initialize vip lite success
Initialize network items success
Create Neural Network: 4.88ms or 4882.67us
Create the neural network success
buf_param.quant_forma = VIP_BUFFER_QUANTIZE_TF_ASYMM !!
Pre process the input/output data success
Start run graph [1] times...
Run the 1 time: 20.76ms or 20756.62us
vip run network execution time:
Total 20.85ms or 20852.92us
Average 20.85ms or 20852.92us
Run the neural network success
data_format=2 buff_size=176400
0: 129%, [ 24, 86, 114, 320], person
5: 129%, [ 5, 12, 314, 265], bus
Post process output data success[34968.930592] npu[c13][c13] gckvip_drv_exit, aw_vip_mem_free
[34968.939459] aw_vip_mem_free vir 0xe31fe000, phy 0x48800000
[34968.946540] npu[c13][c13] vipcore, device un-init..
root@TinaLinux:/mnt/UDISK#

```

运行程序之后会保存输出图像 yolov8_out.jpg。

注意：目前yolov8模型进行uint8量化后。端侧部署时会出现精度下降严重问题，表现为：检测概率值不变，检测效果变差。

解决思路：

- 1.使用int16量化，提高模型精度
- 2.后处理节点跑在CPU，可参考yolov5的后处理代码。